

## Technical Support Ticket Management System

### MÓDULO 9 — Sistema de Comunicación y Cierre del Ciclo de Vida

**Objetivo:** Implementar un sistema de hilos de conversación (comentarios), feedback visual al usuario (notificaciones) y la lógica de cierre definitivo del ticket.

Para cerrar el ciclo de vida del ticket y permitir que los técnicos y empleados colaboren, se usa la clase ***TicketComment***, que ya se tiene definida. Se va a implementar el sistema de mensajes entre el administrador, los técnicos y empleados mediante una interfaz tipo "hilo de conversación" (estilo chat) y la lógica para que el técnico o el administrador puedan dar el cierre definitivo al caso. Esto convertirá al sistema de tickets en una herramienta de colaboración real.

#### 1. Modelo de Datos: Sistema de Comentarios

Primero, se requiere tener una tabla para guardar las conversaciones e intercambios de mensajes.

Verificar que tiene la clase: `/Models/TicketComment.cs`

```
1 namespace SupportTicketsSystem2026.Models;
2
3 7 references public class TicketComment
4 {
5     0 references public int Id { get; set; } // ID del comentario
6     2 references public required string Content { get; set; } // Contenido del comentario
7     3 references public DateTime CreatedAt { get; set; } = DateTime.UtcNow; // Fecha de creación del comentario
8
9     //Relaciones
10    2 references public int TicketId { get; set; } // ID del ticket al que pertenece el comentario
11    0 references public Ticket? Ticket { get; set; } // Relación con el ticket
12
13    3 references public required string UserId { get; set; } // ID del usuario que hizo el comentario
14    2 references public Data.ApplicationUser? User { get; set; } // Relación con el usuario
15 }
16
```

#### Importante (opcional):

1. La clase ***TicketComment*** se trabajó en el módulo 3: **Modelado del Dominio del Sistema de Tickets**. Si no tiene la clase ***TicketComment***, debe crearla.
2. Agregar: `public DbSet<TicketComment> TicketComments => Set<TicketComment>();` en ***ApplicationDbContext***.
3. Ejecutar la migración: ***Add-Migration AddComments*** y ***Update-Database***.

## 2. Lógica del Servicio de Tickets: TicketService.cs

Se agregan los métodos para gestionar la conversación y el cierre del ticket.

### Editar el Archivo: /Services/TicketService.cs

```
149     public async Task<List<ApplicationUser>> GetAllUsersAsync()
150     {
151         using var context = dbFactory.CreateDbContext();
152         return await context.Users.OrderBy(u => u.Email).ToListAsync();
153     }
154
155     // --- MÓDULO 9 – Comunicación y Cierre del Ciclo de Vida ---
156     // Método para agregar un comentario a un ticket
157     public async Task AddCommentAsync(TicketComment comment)
158     {
159         using var context = dbFactory.CreateDbContext();
160         context.TicketComments.Add(comment);
161         await context.SaveChangesAsync();
162     }
163     // Método para obtener los comentarios de un ticket específico
164     public async Task<List<TicketComment>> GetCommentsAsync(int ticketId)
165     {
166         using var context = dbFactory.CreateDbContext();
167         return await context.TicketComments
168             .Include(c => c.User) // Incluir información del usuario que hizo el comentario
169             .Where(c => c.TicketId == ticketId)
170             .OrderBy(c => c.CreatedAt)
171             .ToListAsync();
172     }
173     // Método para cerrar un ticket (cambiar su estado a Resuelto o Cerrado)
174     public async Task CloseTicketAsync(int ticketId)
175     {
176         using var context = dbFactory.CreateDbContext();
177         var ticket = await context.Tickets.FindAsync(ticketId);
178         if (ticket != null)
179         {
180             ticket.Status = TicketStatus.Resolved; // O un estado 'Closed'
181             ticket.UpdatedAt = DateTime.UtcNow;
182             await context.SaveChangesAsync();
183         }
184     }
185 }
186
187
```

## 3. Lógica de Estados en ManageTicket.razor

En el diseño de la aplicación, la comunicación se centraliza en un hilo de conversación único dentro de la página de gestión del ticket. Esto permite que tanto el técnico como el empleado vean la misma "historia" del problema.

Para que el flujo sea natural, los técnicos no solo deben "ver" el ticket, sino tener botones de acción que cambien el estado según lo que están haciendo. En éste **Módulo**, se refina la interfaz de ManageTicket.razor para que los botones aparezcan dinámicamente según el estado actual del ticket, no queremos mostrar todos los botones a la vez, sino solo el siguiente paso lógico.

Cómo interactúan los roles y cómo el Administrador supervisa todo el proceso:

### A. Intercambio de Comentarios (Técnico ↔ Empleado)

La clave es que la página ManageTicket.razor es **compartida**, pero sus funciones cambian según quién la mira.

- **El Técnico:** Entra para dar actualizaciones técnicas ("He reiniciado el servidor", "Necesito acceso remoto").
- **El Empleado:** Entra para responder o dar más detalles ("Ya funciona", "Sigo viendo el error", "Aún no tengo acceso a Internet").

---

## B. ¿Cómo ve el Admin la evolución?

El Administrador tiene "**Visión Total**". Gracias a la directiva de autorización que se puso en el Dashboard (Authorize(Roles = Roles.Admin)), el admin puede hacer clic en "**Manage**" en cualquier ticket del sistema, no solo en los suyos.

Cuando el Admin entra a ManageTicket.razor:

1. **Auditoría:** Ve exactamente qué le dijo el técnico al empleado y cuánto tiempo pasó entre respuestas.
2. **Intervención:** El Admin también tiene la caja de comentarios habilitada. Puede escribir un comentario para corregir al técnico o para informar al empleado que él mismo tomará el caso.
3. **Seguimiento de Estado:** El Admin ve las notificaciones automáticas del sistema (ej: "[System] Status changed to InProgress").

## C. Implementación de la "Línea de Tiempo" (Timeline)

Para que el Admin vea la "evolución", es vital que los comentarios y los cambios de estado estén ordenados en orden cronológico. En el TicketService.cs, asegurarse de que la consulta siempre traiga el historial ordenado:

```
public async Task<List<TicketComment>> GetCommentsByTicketAsync(int ticketId)
{
    using var context = dbFactory.CreateDbContext();
    return await context.TicketComments
        .Include(c => c.Author)
        .Where(c => c.TicketId == ticketId)
        .OrderBy(c => c.CreatedAt) // Cronología perfecta
        .ToListAsync();
}
```

## D. El "Ciclo de Cierre" visto por el Admin

El Admin utiliza el Dashboard (implementado en el Módulo 7) para detectar tickets que llevan mucho tiempo en "InProgress" sin comentarios nuevos.

- Si el Admin ve que un técnico no está avanzando, puede entrar al ticket, usar la pestaña de gestión para **reasignar** el técnico (implementado en el Módulo 6) y dejar un comentario explicando el cambio.
- Una vez que el técnico marca como "**Resolved**", el Admin recibe la actualización visual en sus tarjetas estadísticas del Dashboard.

Abrir y editar el archivo Components/Pages/Ticket/ManageTicket.razor:

```
1  @page "/tickets/manage/{TicketId:int}"
2  @using Microsoft.AspNetCore.Authorization
3  @using System.Security.Claims
4  @inject TicketService TicketService
5  @inject NavigationManager Navigation
6  @inject AuthenticationStateProvider AuthProvider
7  @attribute [Authorize(Roles = $"{Roles.Admin},{Roles.Technician}")]
8  @rendermode InteractiveServer
9
10 <div class="container mt-4">
11     @* --- 1. ZONA DE NOTIFICACIONES --- *@
12     @if (!string.IsNullOrEmpty(feedbackMessage))
13     {
14         <div class="alert alert-success alert-dismissible fade show shadow-sm" role="alert">
15             <i class="bi bi-check-circle-fill me-2"></i> @feedbackMessage
16             <button type="button" class="btn-close" @onclick="()" => feedbackMessage = string.Empty"></button>
17         </div>
18     }
19     @if (ticket == null)
20     {
21         <p><em>Loading...</em></p>
22     }
23     else
24     {
25         <div class="card shadow border-0">
26             <div class="card-header bg-primary text-white d-flex justify-content-between align-items-center">
27                 <h4 class="mb-0">Manage Ticket #@ticket.Id</h4>
28                 <span class="badge rounded-pill bg-light text-dark px-3">@ticket.Status</span>
29             </div>
30             <div class="card-body">
31                 <h5>@ticket.Title</h5>
32                 <p class="text-muted small">
33                     Reported by: <strong>@ticket.Author?.Email</strong> on @ticket.CreatedAt.ToLongDateString()
34                 </p>
35                 <hr />
36                 <p class="fs-5">@ticket.Description</p>
37
38                 @* --- 2. SISTEMA DE COMENTARIOS (HILOS) --- *@
39                 <div class="mt-5">
40                     <h6><i class="bi bi-chat-left-text"></i> Conversation History</h6>
41                     <div class="bg-light p-3 rounded shadow-inner"
42                         style="max-height: 350px; overflow-y: auto; border: 1px solid #dee2e6;">
43                         @if (comments == null || !comments.Any())
44                         {
45                             <p class="text-center text-muted py-3">
46                                 No messages yet. Start the conversation below.
47                             </p>
48                         }
49                         else
50                         {
51                             @foreach (var c in comments)
52                             {
53                                 <div class="d-flex mb-3 @(c.UserId == currentUserId ?
54                                     "justify-content-end" : "justify-content-start")">
55                                     <div class="p-2 px-3 rounded-3 @(c.UserId == currentUserId ?
56                                         "bg-primary text-white" : "bg-white border text-dark")" style="max-width: 75%;">
57                                         <div class="d-flex justify-content-between small opacity-75 mb-1">
58                                             <span class="fw-bold me-2">@c.User?.Email</span>
59                                             <span>@c.CreatedAt.ToShortTimeString()</span>
60                                         </div>
61                                         <div>@c.Content</div>
62                                     </div>
63                                 </div>
64                             }
65                         }
66                     </div>
67                     <div class="input-group mt-2">
68                         <input type="text" class="form-control" placeholder="Type your update here..."
69                         @bind="newCommentContent" />
70                         <button class="btn btn-primary" @onclick="SendComment">
71                             <i class="bi bi-send"></i> Send
72                         </button>
73                     </div>
74                 </div>
75
76                 @* --- 3. SECCIÓN DE ASIGNACIÓN Y ACCIONES --- *@
77                 <div class="row mt-5 bg-light p-4 rounded-3 border">
78                     @* SECCIÓN PARA EL ADMINISTRADOR: ASIGNACIÓN *@
79                     <AuthorizeView Roles="@Roles.Admin">
80                         <div class="col-md-5 border-end">
81                             <h6 class="text-uppercase small fw-bold text-muted mb-3">
82                                 Administrator: Assign Tech
83                             </h6>
84                             <select class="form-select mb-2" @bind="selectedTechId">
85                                 <option value="">-- Select Technician --</option>
86                                 @foreach (var tech in technicians)
87                                 {
88                                     <option value="@tech.Id">@tech.Email</option>
89                                 }
90                             </select>
91                             <button class="btn btn-success w-100 shadow-sm" @onclick="AssignTicket">
92                                 <i class="bi bi-person-plus"></i> Confirm Assignment
93                             </button>
94                         </div>
95                     </AuthorizeView>
96                 </div>
97             </div>
98         </div>
99     }
100 </div>
```

```

97     @* SECCIÓN PARA EL TÉCNICO: CAMBIO DE ESTADO *@
98     @* SECCIÓN DE ACCIONES DINÁMICAS *@
99     <div class="card-footer bg-white p-3">
100         <div class="d-flex justify-content-between align-items-center">
101             <h6 class="mb-0 text-muted">Ticket Actions:</h6>
102
103             <div class="btn-group shadow-sm">
104                 @if (ticket?.Status == TicketStatus.Open)
105                 {
106                     <button class="btn btn-warning" @onclick="() =>
107                         UpdateStatus(TicketStatus.InProgress)">
108                         <i class="bi bi-play-fill"></i> Start Work
109                     </button>
110                 }
111
112                 @if (ticket?.Status == TicketStatus.InProgress)
113                 {
114                     <button class="btn btn-success" @onclick="() =>
115                         UpdateStatus(TicketStatus.Resolved)">
116                         <i class="bi bi-check-all"></i> Mark as Resolved
117                     </button>
118                 }
119
120                 @if (ticket?.Status == TicketStatus.Resolved)
121                 {
122                     <div class="alert alert-info w-100 mb-0 py-2">
123                         <i class="bi bi-info-circle"></i> This ticket is closed.
124                     </div>
125                 }
126
127                 @* El Admin siempre puede re-abrirlo si es necesario *@
128                 <AuthorizeView Roles="@Roles.Admin">
129                     @if (ticket?.Status == TicketStatus.Resolved)
130                     {
131                         <button class="btn btn-outline-secondary" @onclick="() =>
132                             UpdateStatus(TicketStatus.Open)">
133                             <i class="bi bi-arrow-counterclockwise"></i> Re-open Ticket
134                         </button>
135                     }
136                 </AuthorizeView>
137             </div>
138         </div>
139     </div>
140 </div>
141 </div>
142 </div>
143 }
144 </div>

```

```

145 @code {
146     [Parameter] public int TicketId { get; set; }
147     private Ticket? ticket;
148     private List<ApplicationUser> technicians = [];
149     private List<TicketComment> comments = [];
150     private string selectedTechId = "";
151     private string newCommentContent = "";
152     private string currentUserId = "";
153     private string feedbackMessage = "";
154
155     protected override async Task OnInitializedAsync()
156     {
157         // Get the current authenticated user
158         var authState = await AuthProvider.GetAuthenticationStateAsync();
159         var user = authState.User;
160         currentUserId = user.FindFirst(ClaimTypes.NameIdentifier)?.Value ?? "";
161
162         //ticket = await TicketService.GetTicketByIdAsync(TicketId);
163         //technicians = await TicketService.GetTechniciansAsync();
164
165         await LoadData();
166     }
167
168     private async Task LoadData()
169     {
170         ticket = await TicketService.GetTicketByIdAsync(TicketId);
171         technicians = await TicketService.GetTechniciansAsync();
172         comments = await TicketService.GetCommentsAsync(TicketId);
173         if (ticket?.TechnicianId != null) selectedTechId = ticket.TechnicianId;
174     }
175
176     private async Task SendComment()
177     {
178         if (string.IsNullOrEmpty(newCommentContent)) return;
179
180         var comment = new TicketComment
181         {
182             Content = newCommentContent,
183             TicketId = TicketId,
184             UserId = currentUserId,
185             CreatedAt = DateTime.UtcNow
186         };
187
188         await TicketService.AddCommentAsync(comment);
189         newCommentContent = "";
190         comments = await TicketService.GetCommentsAsync(TicketId);
191         feedbackMessage = "Comment posted successfully!";
192     }
193 }

```



```
194     private async Task AssignTicket()
195     {
196         if (string.IsNullOrEmpty(selectedTechId)) return;
197         await TicketService.UpdateTicketStatusAsync(TicketId, TicketStatus.InProgress, selectedTechId);
198         Navigation.NavigateTo("/admin/dashboard");
199     }
200
201     private async Task UpdateStatus(TicketStatus status)
202     {
203         await TicketService.UpdateTicketStatusAsync(TicketId, status);
204         await LoadData();
205         feedbackMessage = $"Ticket marked as {status}.";
206     }
207 }
208
209
```

## Actualización del @code (Variables y Carga)

Se agrega el soporte para el usuario actual (quien escribe) y la lista de comentarios.

- **Inyección:** Se requiere `@inject AuthenticationStateProvider AuthProvider` al inicio para saber quién está logueado.
- **Variables:** Añadir `List<TicketComment> comments`, `string newCommentContent`, `string feedbackMessage` y `string currentUserId`.
- **Inicialización:** En `OnInitializedAsync`, se extrae el ID del usuario del `AuthenticationState` y se llama a un nuevo método `LoadComments()`.

## Estructura de la Interfaz (HTML)

**Zona de Alertas:** Justo arriba del `if (ticket == null)`, colocaremos un bloque que muestre la `feedbackMessage` usando clases de Bootstrap (`alert-success`), con un botón para cerrarla manualmente.

## Lógica de Métodos (Funcionalidad)

- **UpdateStatus mejorado:** Al final de la ejecución, se asigna un mensaje a `feedbackMessage` (por ejemplo: *"¡Estado actualizado correctamente!"*).
- **Nuevo método SendComment:** Validará que el texto no esté vacío, creará el objeto `TicketComment` con el `UserId` actual y el `TicketId`, lo enviará al servicio y limpiará el cuadro de texto.
- **Cierre definitivo:** El botón **Mark as Resolved** funciona como cierre. Se añade un mensaje que confirme que el ciclo de vida del ticket ha terminado al llegar a ese estado.

## Actualizar Repositorio (Requerido)

```
Developer PowerShell
+ Developer PowerShell v |  |  |
                                git status

On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   SupportTicketsSystem2026/Components/Pages/Tickets/ManageTicket.razor
    modified:   SupportTicketsSystem2026/Models/TicketComment.cs
    modified:   SupportTicketsSystem2026/Services/TicketService.cs

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status

On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   SupportTicketsSystem2026/Components/Pages/Tickets/ManageTicket.razor
    modified:   SupportTicketsSystem2026/Models/TicketComment.cs
    modified:   SupportTicketsSystem2026/Services/TicketService.cs

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Modulo 9: Implementar la logica de un sistema de hilos de conversación, feedback y la lógica de cierre d
[main c1e534b] Modulo 9: Implementar la logica de un sistema de hilos de conversación, feedback y la lógica de cierre definitivo del ticket
 3 files changed, 183 insertions(+), 28 deletions(-)
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 24 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 3.91 KiB | 3.91 MiB/s, done.
Total 11 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 8 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
 dcl5e0e..c1e534b  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```

Ejecutar la aplicación

Ticket System

Home

Register

Login

About

# Welcome to the Support Portal

Current Status: System Online

Today is: Saturday, 04 April 2026

This platform allows employees to report incidents and track technical support in real-time.

Login Admin:

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

(admin@system.com)

About

# Welcome to the Support Portal

Current Status: System Online

Today is: Saturday, 04 April 2026

This platform allows employees to report incidents and track technical support in real-time.

Admin ➔ My Tickets

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

(admin@system.com)

About

# My Ticket List

| Title                       | Status     | Priority | Action     |
|-----------------------------|------------|----------|------------|
| Programado que no funciona  | InProgress | High     | Work on it |
| Internet Lento              | Resolved   | Urgent   | Work on it |
| Pantalla no se ve           | Open       | High     | Work on it |
| Fallo en el mouse           | Open       | Medium   | Work on it |
| Cuenta bloqueada            | Open       | High     | Work on it |
| Posible infección por virus | Open       | Urgent   | Work on it |

Admin ➔ Dashboard

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

(admin@system.com)

About

# Administrative Dashboard

Real-time Support Metrics

TOTAL TICKETS6

OPEN / IN PROGRESS5

RESOLVED1

URGENT CASES2

All Support Requests

| Title                       | Author               | Category | Status     | Technician          | Actions |
|-----------------------------|----------------------|----------|------------|---------------------|---------|
| Posible infección por virus | empleado6@system.com | Software | InProgress | tecnico2@system.com | Manage  |
| Cuenta bloqueada            | empleado4@system.com | Network  | InProgress | tecnico3@system.com | Manage  |
| Fallo en el mouse           | empleado7@system.com | Hardware | InProgress | tecnico1@system.com | Manage  |
| Pantalla no se ve           | empleado3@system.com | Hardware | Open       | tecnico2@system.com | Manage  |
| Internet Lento              | empleado2@system.com | Network  | Resolved   | tecnico1@system.com | Manage  |
| Programado que no funciona  | empleado1@system.com | Software | InProgress | tecnico2@system.com | Manage  |

Admin ➔ Dashboard ➔ Manage ➔ Assign Tech

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

<|

(admin@system.com)

About

Manage Ticket #4

Open

Fallo en el mouse

Reported by: empleado7@system.com on Saturday, April 4, 2026

No logro ver el puntero del mouse, no funciona, pero si puedo usar el teclado.

Conversation History

No messages yet. Start the conversation below.

Type your update here...

Send

ADMINISTRATOR: ASSIGN TECH

-- Select Technician --

-- Select Technician --

tecnico2@system.com

tecnico3@system.com

tecnico1@system.com

Start Work

Login Técnico ➔ My Tickets

Ticket System

Home

New Ticket

My Tickets

Logout

<|

(tecnico2@system.com)

About

My Ticket List

| Title                        | Status     | Priority | Action     |
|------------------------------|------------|----------|------------|
| Programado que no funciona   | InProgress | High     | Work on it |
| Pantalla no se ve            | Open       | High     | Work on it |
| Possible infección por virus | InProgress | Urgent   | Work on it |

Técnico ➔ My Tickets ➔ Manage Ticket

Ticket System

Home

New Ticket

My Tickets

Logout

<|

(tecnico2@system.com)

About

Manage Ticket #3

Open

Pantalla no se ve

Reported by: empleado3@system.com on Monday, March 23, 2026

Problemas de visualización: Pantallas negras y cuando enciende se ven colores distorsionados o falta total de señal de video.

Conversation History

No messages yet. Start the conversation below.

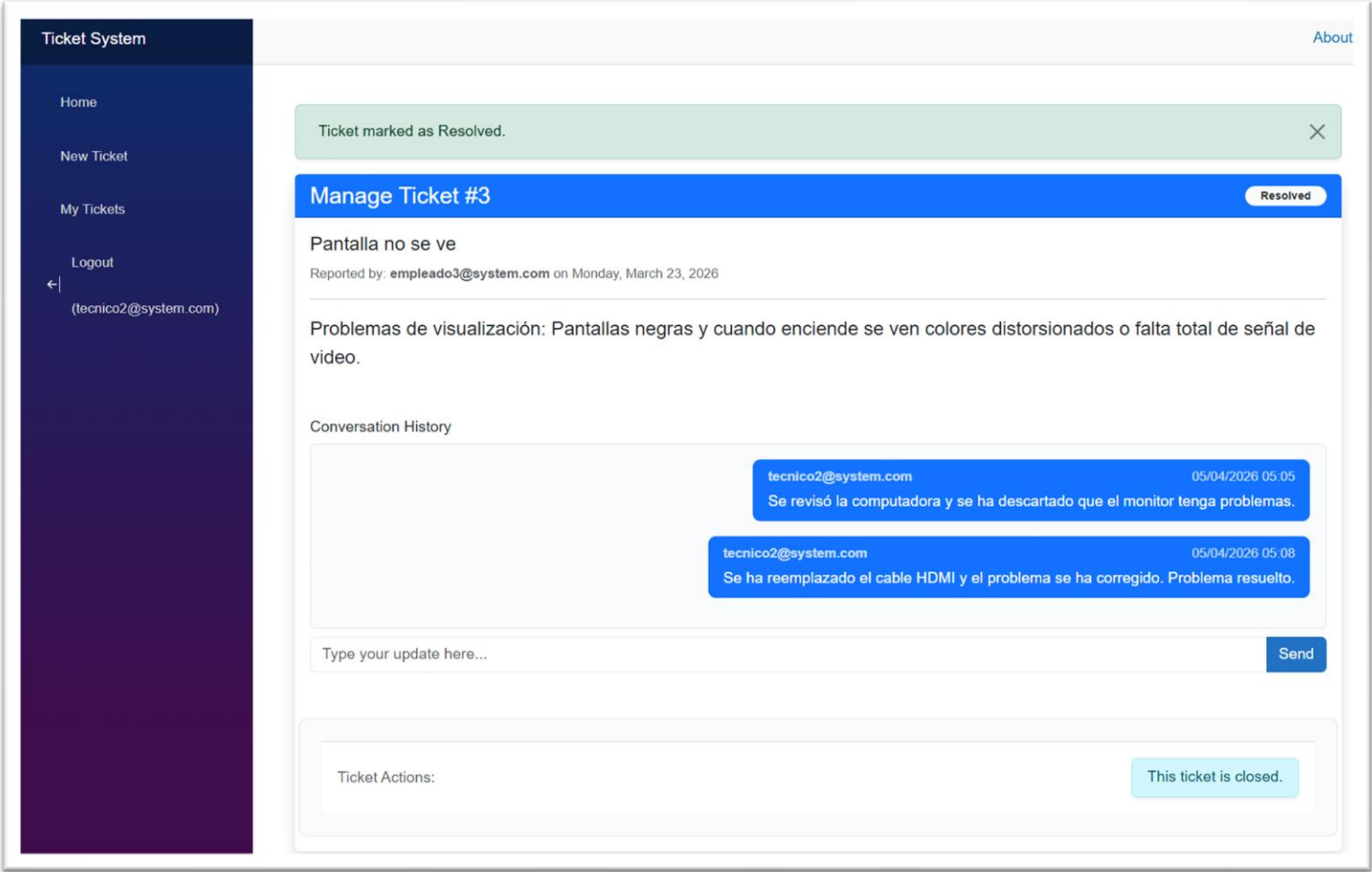
Type your update here...

Send

Ticket Actions:

Start Work





**Hilo de Conversación:** Debajo de la descripción del ticket, pero antes de la sección de acciones, se tiene:

- Un bucle @foreach que recorre comments.
- Lógica condicional para alinear a la derecha los mensajes del currentUserId (azul) y a la izquierda los demás (gris/blanco).
- Un input-group al final para escribir los comentarios y enviar.

